

LINUX BASICS



Contents

Kernel

Shell

Linux Filesystem Hierarchy

Run Level

Linux Booting Procedure

Basic Commands

- Text Processing Commands
- Archives and Compressions
- System states

OVERVIEW OF LINUX SYSTEM

Linux

Linux is a clone of the operating system Unix, written from scratch by Linus Torvalds with assistance from a loosely-knit team of hackers across the Net.

Kernel

Linux Kernel is the core component of GNU/Linux Operating System. Some of its functions include Process Management, Memory Management, File System, Device Drivers, I/O Management and Networking.

The complete source code of the Linux Kernel can be freely downloaded from <http://kernel.org/>

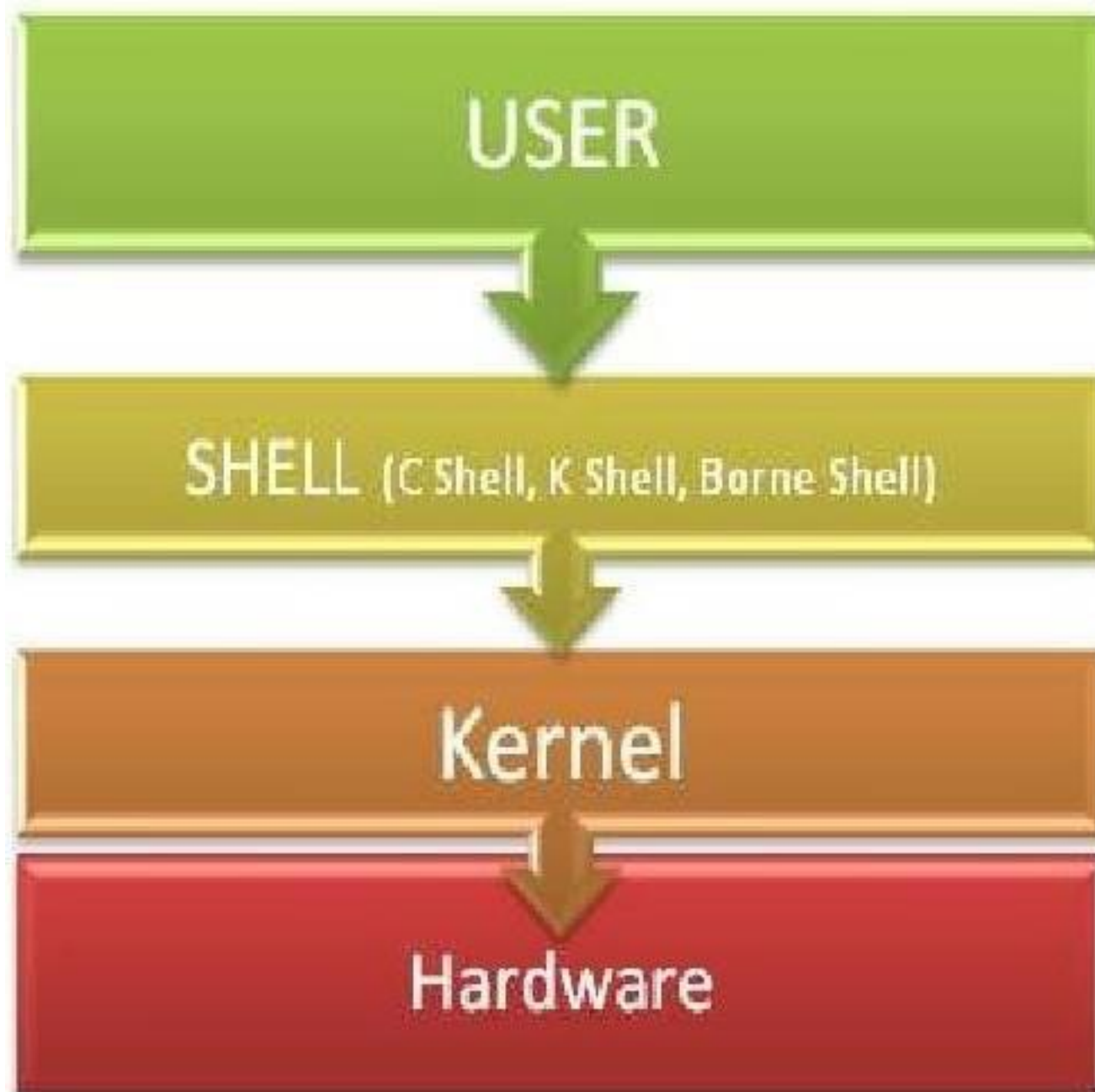
Shell

Shell is an interface between user and kernel. Shell interprets your input as commands and pass them and pass them to kernel.

Common shells in popular Linux Distributions include C Shell, Sh, K Shell, Bash

- A program that interprets commands
- Allows a user to execute commands by typing them manually at a terminal, or automatically in programs called shell scripts.
- A shell is not an operating system. It is a way to interface with the operating system and run commands.

LINUX COMMAND FLOW



FEATURES OF LINUX OS

o **Multi-user & Multi- tasking**

- o Supports full multitasking and multiuser functionalities.

o **Files and Process**

- Data, directory, processes and Hardware, almost everything are expressed as a file.
- Process is an running program identified by a unique id (PID).

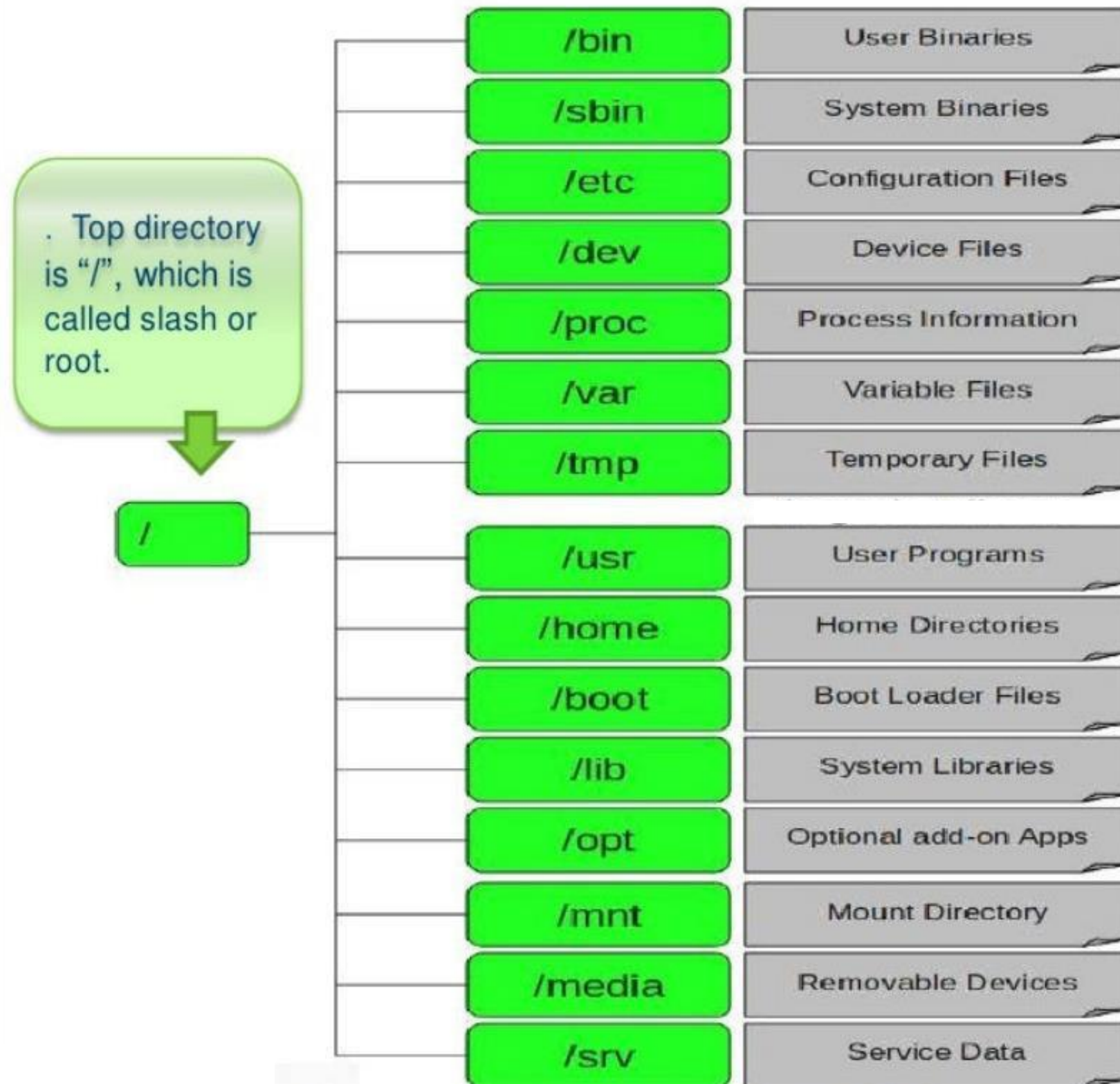
o **Case Sensitivity**

- Linux like UNIX is case- sensitive (MYFILE.txt, Myfile.txt, mYfile.txt all are different)

NORMAL USER AND SUPER USER

- In Linux system, one special user is created by default during installation which has almost unlimited power. This special user is called root or super user.
 - Conventionally, a normal User has the shell prompt ending in '\$' sign.
 - Super User shell prompt ends in '#'

Linux Filesystem Hierarchy



RUN LEVEL

The init process, by default, runs the system in one of the eight runlevels.

	Run Level
Runlevel 0	Halt System - To shutdown the system
Runlevel 1	Single user mode
Runlevel 2	Basic multi user mode without NFS
Runlevel 3	Full multi user mode (text based)
Runlevel 4	Unused
Runlevel 5	Multi user mode with Graphical User Interface
Runlevel 6	Reboot System

LINUX BOOTING

BIOS

MBR

GRUB

Kernel

Init

Runlevel

BIOS

- BIOS stands for Basic Input/Output System
- Performs some system integrity checks
- Searches, loads, and executes the boot loader program.
- It looks for boot loader in floppy, cd-rom, or hard drive. You can press a key (typically F12 or F2, but it depends on your system) during the BIOS startup to change the boot sequence.
- Once the boot loader program is detected and loaded into the memory, BIOS hands over the control to it.

MBR

- MBR stands for Master Boot Record.
- It is located in the 1st sector of the bootable disk. Typically /dev/hda, or /dev/sda
- MBR is less than 512 bytes in size. MBR has three components
 - 1) Primary boot loader info in 1st 446 bytes
 - 2) Partition table info in next 64 bytes
 - 3) MBR validation check in last 2 bytes.
- MBR holds the first stage of GRUB boot loader (or LILO - Linux Loader in older systems).

GRUB

- GRUB stands for GNU Grand Unified Boot loader.
- GRUB is dynamically configurable. It loads its configuration at startup, allowing boot-time changes, such as selecting different kernels or initial RAM disks
- GRUB can display splash screen. It loads the default kernel image as specified in the grub configuration file.
- GRUB is highly portable. It supports multiple executable formats and is geometry translation independent.
- GRUB supports operating systems that do not multiboot (MS Windows), by using chain loading.

KERNEL

- Mounts the root file system as specified in the "root=" directive in the grub.conf file
- Root file-system can also be specified in the GRUB prompt.
- Kernel executes the /sbin/init program
- Since init is the first program to be executed by Linux Kernel, it has the process id (PID) of 1. Do a 'ps -ef | grep init' and check the pid.
- initrd stands for Initial RAM Disk.
- initrd is used by kernel as temporary root file system until kernel is booted and the real root file system is mounted. It also contains necessary drivers compiled inside, which helps it to access the hard drive partitions, and other hardware.

INIT

- Looks at the /etc/inittab file to decide the Linux run level (In Red Hat Linux).
- Following are the available run levels (In Red Hat Linux)
 - 0 - halt
 - 1 - Single user mode
 - 2 - Multiuser, without NFS
 - 3 - Full multiuser mode
 - 4 - unused
 - 5 - Graphical environment
 - 6 - reboot
- Init identifies the default initlevel from /etc/inittab and uses that to load all appropriate program.
- Execute 'grep initdefault /etc/inittab' on your system to identify the default run level
- If you want to get into trouble, you can set the default run level to 0 or 6. Since you know what 0 and 6 means, probably you might not do that.
- Typically you would set the default run level to either 3 or 5.

RUNLEVELPROGRAMS

- When the Linux system is booting up, various services getting started. Those are the runlevel programs, executed from the run level directory as defined by your run level.
- Depending on your default init level setting, the system will execute the programs from one of the following directories.
 - Run level 0 - /etc/rc.d/rc0.d/
 - Run level 1 - /etc/rc.d/rc1.d/
 - Run level 2 - /etc/rc.d/rc2.d/
 - Run level 3 - /etc/rc.d/rc3.d/
 - Run level 4 - /etc/rc.d/rc4.d/
 - Run level 5 - /etc/rc.d/rc5.d/
 - Run level 6 - /etc/rc.d/rc6.d/

BASIC COMMANDS

- o man - format and display the on-line manual page

man date

- o help - Display help for a built-in command

date -help

- o info - information about the command ('info' utility must be installed)

info date

Basic Commands

- o `ls` (`ls` lists the content of directory)
- o `ls -l` list the files with all information
- o `ls -a` list files including hidden files
- o `ls -la` long list with hidden files

Check 'man' page for all available options.

- o cd Change Directory

- cd /home/user/

0

- o cd home/user/myfile.jpg,
- o cd .. goes one directory back
- o cd - return to the last directory
- o cd ~ go to the current user's home directory

- Absolute path /home (absolute path: Relating to file systems, the location of a directory or file that can be accessed regardless of the current working location of a user)

- `cd /home/user/mypicture.jpg`

- A relative pathname does not begin with a slash (/). Generally you specifies location relative to your current working directory

- `cd user/mypicture.jpg`

- o mkdir Create one or more directories

```
mkdir folder1 folder2
```

```
mkdir -p work/junk/questions work/junk/answers (mkdir -p Create intervening  
parent directories if they don't exist)
```

- o **pwd** **Print the full pathname of the current working directory.**

See also the `dirs` shell command built into `bash`.

```
Pwd
```

- o **cp** **Copy files/directories**

```
cp file1 file2
```

```
cp -vrf dirA dirB
```

- o **mv** **move files to another location or rename files**

```
mv /home/user/myfile /home/Admin/myfile
```

```
mv myfile your file
```

- o **rm remove files/directories**

rm myfile

rm -rf mydirectory (rmdir remove directory)

- o **find search for files in a directory hierarchy,**

find /home -name myfile

- o **history shows the history of commands**

History

-c Clear the history list

Start a history substitution, except when followed by a space

!n Refer to command line n.

!-n Run the command n lines back.

!! Run the previous command.

- o **mount** **mounts a file system**

- o The file structure on device is mounted on directory. If no device is specified, mount looks for an entry in /etc/fstab to find out what device is associated with the given directory.

`mount /dev/sdb1 /media`

`mount -t` to limit this to all filesystems of a particular type.

- o **umount** allows to unmount the mounted filesystems

`umount /dev/sdb1` or `umount /media`

- o **fdisk** Linux command line disk partitioning tool
- o **du** Estimate file space usage
- o **df** shows the disk free
- o **dd** Convert and copy a file, write disk headers, boot records, create a boot floppy. dd can make an exact clone of an disk, this will include all blank space so the output destination must be at least as large as the input. dd if=linux_cd_image.iso of=/dev/sdb

- o **touch** to create empty file or updating the date and time to the current time
- o **diff** Display the differences between two files
- o **clear** Clear terminal screen
- o **echo** display a line of text
echo "hello world" (displays hello world in command line)

wc - print the number of newlines, words, and bytes in files

```
wc file1.txt
```

grep - print lines matching a pattern

```
ls /etc | grep -i network
```

MOST POPULAR TEXT PROCESSING COMMANDS

cat

Read one or more files and print them on standard output.

`cat filename.txt` (display the content of filename.txt)

`cat -n myfiles.txt` number all output lines, starting with 1

`cat> newfile` to exit, enter EOF (Ctrl-D)

`cat >> newfile` to append the file

VI EDITOR

- o External Link **12**
- o A screen-oriented text editor. Vi has two modes insert mode and command mode.

#vi filename (creating or opening a filename)

esc + i insert the text

:wq Save and exit

:q exit without save

:wq! save and exit forcefully (! forcefully)

Formatting in command line

```
mke2fs /dev/sda1
```

```
mkfs.ext3 /dev/sda1
```

```
mkfs.ext4 /dev/sda1
```

TAR

was initially developed to write data to sequential 1/0 devices for tape backup purposes. It is now commonly used to collect many files into one larger file for distribution or archiving

```
tar -cvf mytarball.tar file1 file2 file3
```

```
tar -xvf mytarball.tar
```

o **System states**

- o shutdown init 0, poweroff
- o Reboot init 6, reboot
- o Sleep echo mem > /sys/power/state
- o Hibernation echo disk > /sys/power/state

cat and echo

Use cat to display file contents to the terminal:

```
$ cat bears  
$ cat bears7  
$ cat bears bears7
```

"cat" is short for concatenate. The "cat" command takes one or more files and concatenates their contents to standard output.

echo is used to output arbitrary text to the terminal:

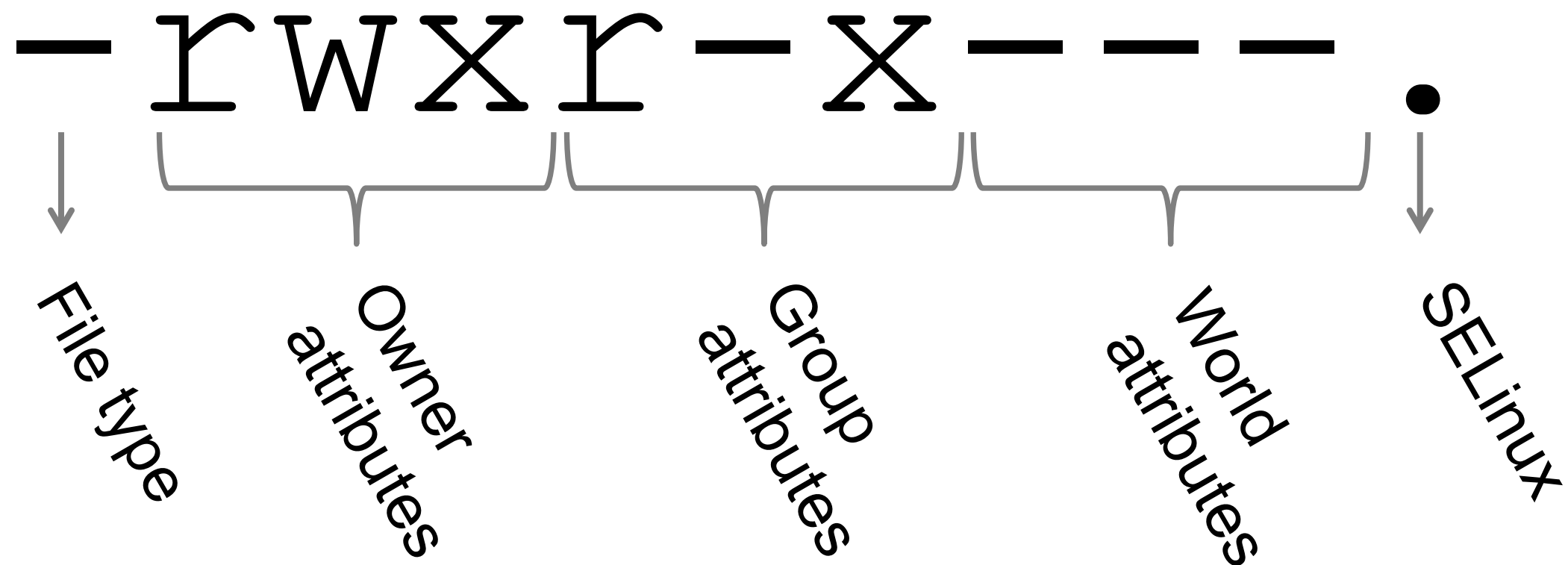
```
$ echo 'Hello World!'  
$ echo without single quotes  
$ echo 'Hello World!' > MyWorld
```

Ownership & Permissions

- ▶ Linux systems are multi-user environments that allow users to create files, run programs and share data.
- ▶ Files and directories have two types of ownership – the user and group. A Linux group consists of one or more users.
- ▶ Files and directories have three types of access permissions:
 - read permission (r)
 - write permission (w)
 - execute permission (x)
- ▶ Every file and directory has permissions for three levels or entities of permissions:
 - a) user or owner (denoted by u)
 - b) group (one or more users – denoted by g)
 - c) others or world (denoted by o)

Permissions triplets

Each triplet indicates the access permissions for that level – in the example below, the user/owner has read, write & execute permission, other group members only have read and execute permissions and all others have no access permissions.



Long List Output Explained

permissions	user	group	size	date		file/directory
drwxr-xr-x	2 paul	users	1024	Jan	2 23:50	.
drwxr-xr-x	6 root	root	1024	Jan	2 22:51	..
drwxr-xr-x	3 paul	users	1024	Jan	8 11:42	grassdata
lrwxrwxrwx	1 paul	users	13	May	6 1998	latex -> /d2/lt
drwx-----	2 paul	users	1024	Mar	8 17:30	mail
drwx-----	2 paul	users	1024	Feb	4 01:09	projects
-rw-r--r--	1 paul	users	844344	Dec	9 1998	nations.ps
-rw-rw-r--	1 paul	users	21438	Mar	2 21:47	ps4mf.txt

other (world) permissions

group permissions

user permissions

d : directory
- : file
l : link (to other file/directory)

r : read permission
w : write permission
x : execute permission (programm)
- : permission not set

Special Directories:

- . is the current working directory
- .. is the “parent” directory, one level “back”

Permissions described:

File Type:

- “-” regular file
- “d” directory
- “l” symlink

Permissions

- “r” read
- “w” write
- “x” execute

Changing Permissions and Ownership

Use '**chmod**' to change the file permissions:

```
chmod [ugoa][+/-][rwx] filename
```

where u=user, g=group, o=others or world and a=all three

For example, to provide group read access to a file:

```
$ chmod g+r myfile
```

Or to remove file access to another than the owner or group members (in other words, others):

```
$ chmod o-rwx myfile
```

The '**chown**' command is used to change file ownership and the '**chgrp**' command can change group ownership of a file. **As a regular user, you can not change the ownership of a file, but you can change the group ownership if you are a member of the group to which you are changing the group ownership**

You can use the **-R** argument on any of the above to recursively make changes on a directory of files

Numeric mode

Example index.html file with typical permission values:

```
$ chmod 755 index.html
$ ls -l index.html
-rwxr-xr-x  1 root  wheel  0 May 24  06:20 index.html
```

```
$ chmod 644 index.html
$ ls -l index.html
-rw-r--r--  1 root  wheel  0 May 24  06:20 index.html
```

drwxrwxrwx

d = Directory
r = Read
w = Write
x = Execute

chmod 777

rwX | rwX | rwX
Owner | Group | Others

7	rwX	111
6	rw-	110
5	r-X	101
4	r--	100
3	-wX	011
2	-w-	010
1	--X	001
0	---	000

Numeric mode

Example index.html file with typical permission values:

```
$ chmod 755 index.html
$ ls -l index.html
-rwxr-xr-x  1 root  wheel  0 May 24  06:20 index.html
```

```
$ chmod 644 index.html
$ ls -l index.html
-rw-r--r--  1 root  wheel  0 May 24  06:20 index.html
```

drwxrwxrwx

d = Directory
r = Read
w = Write
x = Execute

chmod 777

rwX | rwX | rwX
Owner | Group | Others

7	rwX	111
6	rw-	110
5	r-X	101
4	r--	100
3	-wX	011
2	-w-	010
1	--X	001
0	---	000

Wildcards

With many Linux commands, you can use wildcards to match characters

The '*' can be used to match **zero or more** characters

Examples:

```
$ ls bear*
```

```
bears bears7 bears_chicago
```

```
$ ls *bear*
```

```
bears bears7 bears_chicago polarbears
```

The '?' can be used to match **EXACTLY one** character

```
$ ls bears?
```

```
bears7
```

Ctrl-c, ESC & command line editing

If you get into a situation where you can't seem get back to a command line prompt, try hitting the Ctrl-C combination or the Esc key – often one of those will abort whatever you are currently doing.

The Ctrl character is often represented by the ^

Use Ctrl-a to go to the beginning of the line

Use Ctrl-e to go to the end of the line

Output Redirection to Files

Redirect output (>):

```
$ cat bears > Teddybears
$ cat Teddybears
$ cat bears bears7 > bothbears
$ cat bothbears
$ cat bears* > allbears

$ echo 'Hi there!' > greeting
$ cat greeting
```

Append files (>>):

```
$ echo 'Hi yourself!' >> greeting
$ cat Teddybears >> greeting
$ cat greeting
```

Deleting files/directories

Using rm and rmdir

To remove a file:

```
$ rm my_data_file  
$ touch myFile  
$ chmod u-rwx myFile  
$ rm myFile  
$ rm -f myFile
```

To remove a directory:

```
$ rmdir Mydirectory  
$ rm -r 2017
```

rmdir only works if the directory is empty!

Dangerous:

```
$ rm -rf *
```

cp - copying files/directories

cp - copy one or more files or directories

Syntax: cp source destination

```
$ cp football football2
```

```
$ cp -p football football3
```

```
$ ls -la football*
```

```
$ cp -p Sports/footballtee .
```

```
$ mkdir -p Sports/fall
```

```
$ cp -p football2 Sports/fall
```

```
$ cd Sports/fall
```

```
$ cp -p football2 ../football4
```

```
$ cd ..
```

```
$ ls -R Sports
```

Archival copy:

```
$ cp -a Sports Sports2017
```

```
$ cp -pr Sports Sports2018
```

grep – pattern matching search of a file

“grep” – global/ regular expression/ print

```
$ grep cat nonsense.txt
$ grep dog nonsense.txt
$ grep -i dog nonsense.txt # case insensitive
$ grep -v dog nonsense.txt # exclude 'dog'

$ grep --help
$ man grep
```

find – where are my files?

find - used to locate files based on various criteria

```
$ find [path_to_search] [expression/options]
```

```
$ find . -name "*.txt"
```

```
$ man find
```

Thank You